Building Web and Mobile Apps WS2025/26 (Prof. Dr. Cathrin Möller)

Exercise 5 - 24./25.11.2025

Do not hesitate to ask questions or if you want feedback for your coding style.

Please commit and push to GitLab regularly and ensure *@cathrin.moeller* and my two tutors @pinky-jitendra.tinani and @rahul.patil are invited as a reporter.

Learning goals for today

Today we want to get started using **Angular** as a more complex frontend framework. It is popular in enterprise companies as it is complete - in terms of functionality.
We want to create a new application from scratch, generate a few **components** and a **service** and work with the **Router**.
We will learn how to display components all the time (Header) versus on a route condition only and how to fetch **Path Parameters** from the route in the component.
We will practice templating and how to connect a component with a service.

Practical Exercises

1) Create a folder „angular“ in your repo next to the „backend“ to **create a new Angular application** using the latest version of Angular (v20). Navigate to that folder.
   ! Info: the lab PCs might have a node version which is too old, in that case it is fine to use Angular v18 instead. Code should be the same for the exercises. (node -v => v22.7.0 and we would need ^22.12.0)
   To install the Angular CLI and create an app you can either run:

- `npm install -g @angular/cli@20`
- `ng new my-app`

or use npx in case of limited installation rights:

- `npx -p @angular/cli@20 ng new my-app`

- pick SCSS, deny the request to add SSR and do not create a zoneless app
- do not include AI
- navigate to your app => `cd my-app`
- and run the app => `npm start`
- visit localhost:4200 to see it running

2) Now **add some components and a service** to your app using the CLI (v20).

```
ng generate component dashboard-component
ng generate component header-component
ng generate component users-component
ng generate component user-component
ng generate service user-service
```

```
If you use Angular v18 you need to type
ng generate component dashboard
ng generate component header
ng generate component users
ng generate component user
ng generate service user
```

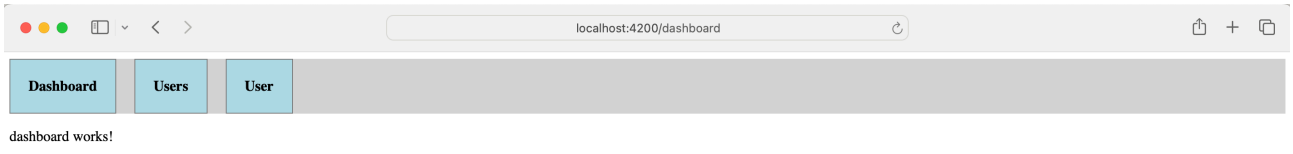
and now **add the HeaderComponent** to your app.html (v20) / app.component.html (v18) template. Remove all the placeholder content except the <router-outlet>

3) Now **we need the router** to conditionally render the content of the components dashboard, users and user.
Add routes with a named path for those 3 components in the app.routes.ts file and also **add router links inside the header template**. You can do so like this if you import the RouterLink directive in your HeaderComponent:

```
<a routerLink="/dashboard">Dashboard</a>
```

Run your app and see you can navigate and that the URL is changing.

Result should look similar to this:



4) Now let's **add users and address data.** You can copy paste the **interfaces** for **„User“** and **„Address“** from your vanilla typescript code to a file named e.g. **user.interface.ts.** With Angular taking care of pre-set typescript configuration, you can easily define and export interfaces in one ts file and import them is any other Typescript file. Be careful with naming conflicts. This would happen if you name the interface the same as any other object, like a component or service class. In that case you might need to refactor/ rename the service or component class. Create a hardcoded array of type User[] (2-5 entries are enough) in the **user.service.ts** file. You will need to import the interface named User. The hardcoded users can be a local state attribute of the service:

```
export class UserService {

users: User[] = [
  {
    id: 1,
    firstname: „John“,
…
```

Implement a get method in the service to be able to fetch the users from a component. We will use this method later.

```
getUsers(): User[] {
```

5) Now use that service from a component and call it using the **ngOnInit** lifecycle method from the **UsersComponent**.

Add the **UserService** as a **dependency** in the **UsersComponent constructor**, create a local state attribute „users“ in the component and copy the return value of the service call into it. You will need to import the interface „User“ here as well, to be able to create a typed array of users as an attribute of the component.

Add the **JsonPipe** to the component:

```
imports: [
```

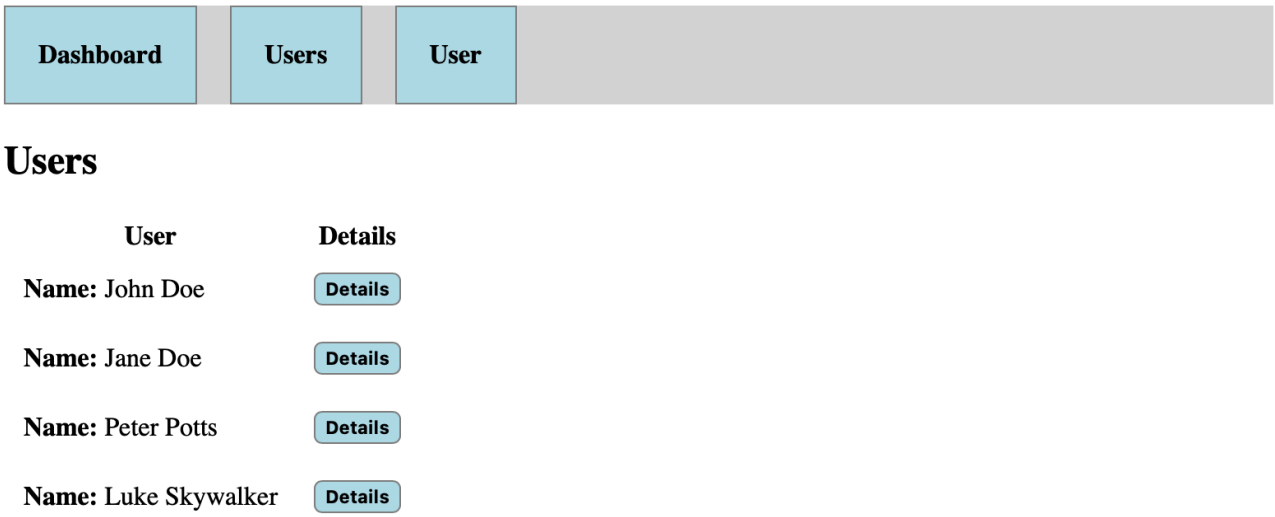
```
  JsonPipe
],
```

Render the users in the template as JSON like this:

```
{{users | json}}
```

6) Now **enhance the representation** of the users in our UsersComponent template. Use ***@for*** to iterate over the users and render all users with their name only in a table. Add a ***routerLink*** wrapped in a button for each entry that is navigating to /user/userId. To add dynamic ids in the routerLink you will need to set it using *[routerLink]="'/user/' + user.id"* . We will add the link mapping later.

The result should look similar to this:



7) Now **add the path parameter userId to the route config** for your UserComponent to be able to show the UserDetailPage of that single user.

You can do so like this:

```
{ path: 'user/:userId', component: UserComponent },
```

And retrieve the value in the component:

```
constructor(private route: ActivatedRoute
```

```
ngOnInit() {
  this.route.paramMap.subscribe(params => {
    this.userId = params.get('userId');
```

Print the userId in the template of the UserComponent.